

## Ejercicio 1 — Más capas en el perceptrón multicapa (Iris)

Enlaces de notebooks en colab

Multilayer:

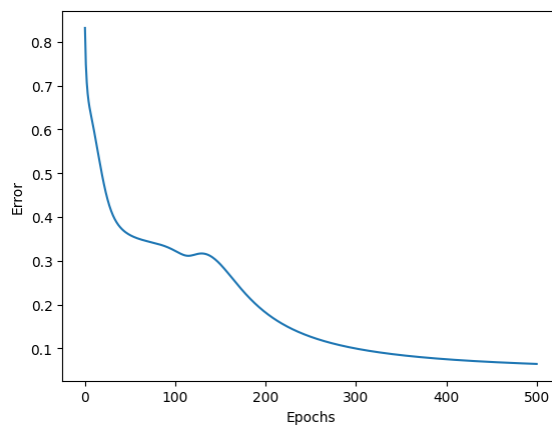
[https://colab.research.google.com/drive/1V3CWVUg0iSHT\\_bkPMGBfZYygc8ch826H#scrollTo=1pQB11m70xfb](https://colab.research.google.com/drive/1V3CWVUg0iSHT_bkPMGBfZYygc8ch826H#scrollTo=1pQB11m70xfb)

Keras:

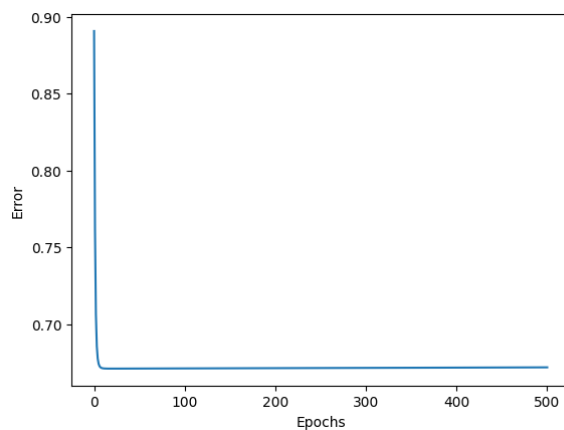
[https://colab.research.google.com/drive/1tcW5YjCOQQVZ\\_Bms\\_X8rsDvwT80PHdRx#scrollTo=W0qUqTvcHXo6](https://colab.research.google.com/drive/1tcW5YjCOQQVZ_Bms_X8rsDvwT80PHdRx#scrollTo=W0qUqTvcHXo6)

### Curvas de error

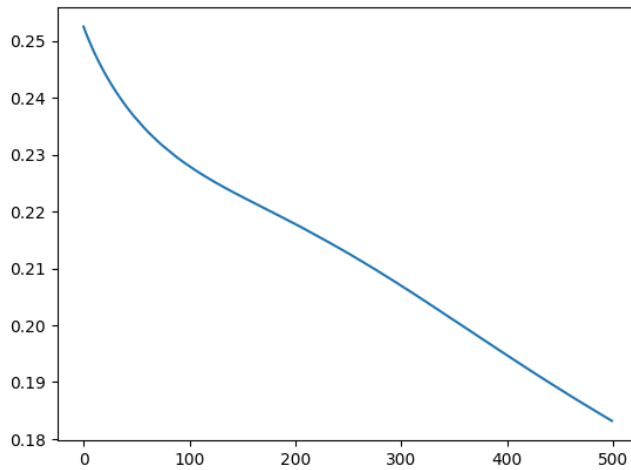
- Curva de error - NumPy original



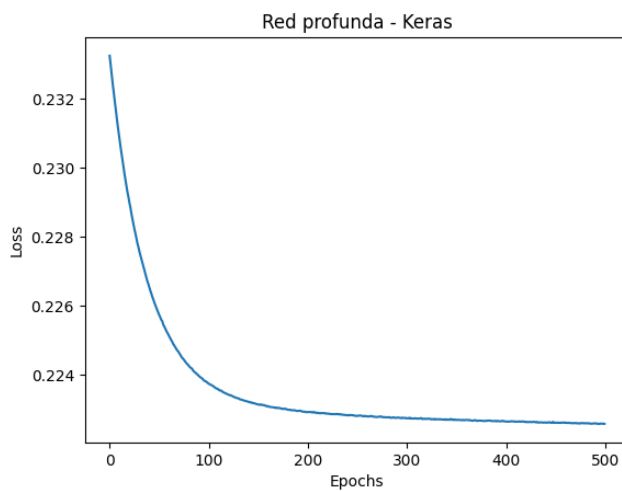
- Curva de error - NumPy profunda



- Curva de pérdida - Keras original



- Curva de pérdida - Keras profunda



- Resumen del modelo Keras original

| Layer (type)   | Output Shape | Param # |
|----------------|--------------|---------|
| layer1 (Dense) | (None, 3)    | 15      |
| layer2 (Dense) | (None, 3)    | 12      |

Total params: 27 (108.00 B)  
 Trainable params: 27 (108.00 B)  
 Non-trainable params: 0 (0.00 B)

- Resumen del modelo Keras profundo

| Layer (type)   | Output Shape | Param # |
|----------------|--------------|---------|
| layer1 (Dense) | (None, 3)    | 15      |
| layer2 (Dense) | (None, 3)    | 12      |
| layer3 (Dense) | (None, 3)    | 12      |
| layer4 (Dense) | (None, 3)    | 12      |

Total params: 51 (204.00 B)  
 Trainable params: 51 (204.00 B)  
 Non-trainable params: 0 (0.00 B)

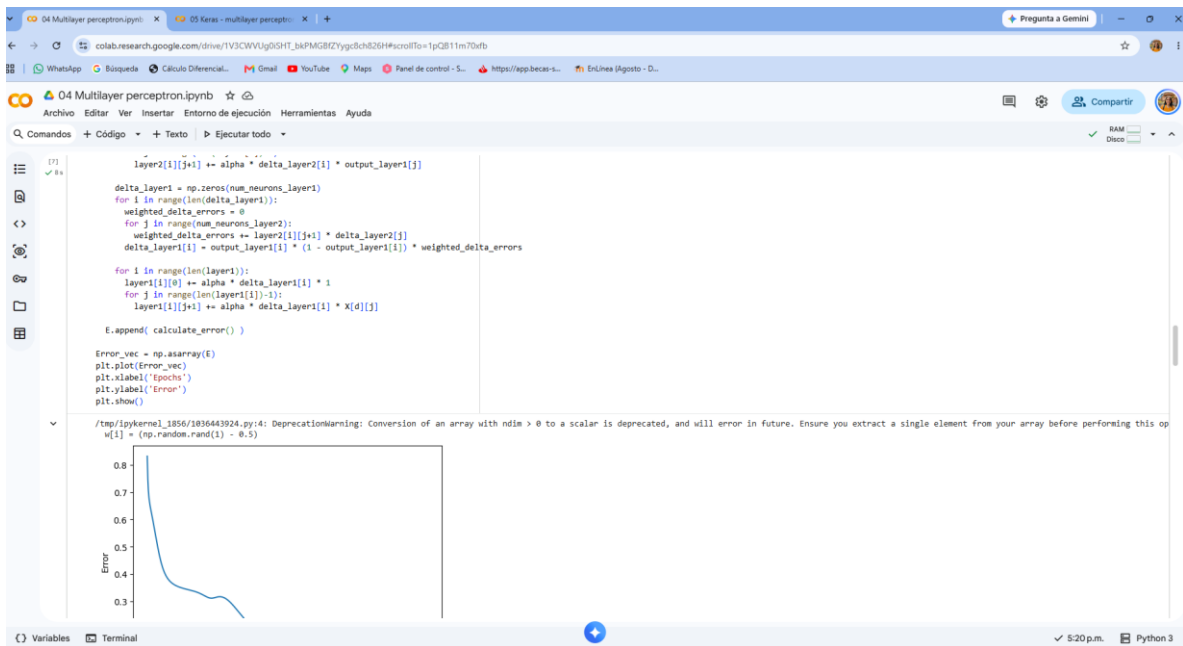
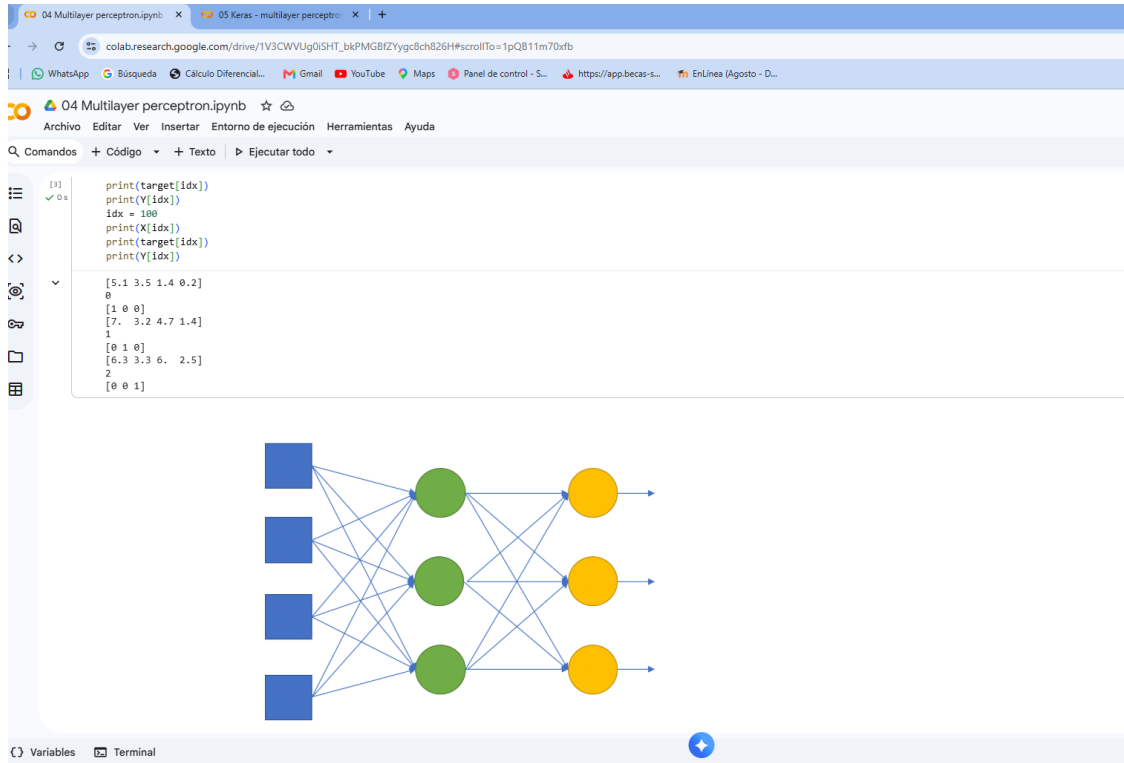
## Reporte

Al comparar las redes, pude observar que agregar dos capas no hizo que el error bajara más, sino que ocurrió lo contrario. En la implementación manual con NumPy, la red original logró disminuir el error aproximadamente hasta 0.07, mientras que la red profunda bajó rápidamente al inicio pero después se quedó prácticamente estancada alrededor de 0.67. En Keras pasó algo parecido: la red original terminó con un loss de aproximadamente 0.183, mientras que la profunda terminó cerca de 0.223 y su curva se fue aplanando. Por lo tanto, en ambas implementaciones la red más profunda no tuvo un mejor resultado.

Las curvas de NumPy y Keras tampoco son exactamente iguales aunque tengan la misma topología y parámetros generales. Esto tiene sentido porque la implementación interna es diferente. En NumPy estamos haciendo manualmente la propagación y la retropropagación, además de recorrer los datos uno por uno y actualizar los pesos de esa manera. Keras, en cambio, trabaja de forma vectorizada y maneja internamente aspectos como los lotes y la actualización de los pesos. También influye que los pesos se inicializan aleatoriamente, por lo que incluso al ejecutar nuevamente el mismo modelo podemos obtener una curva un poco diferente. El orden en que se procesan los datos también puede afectar el entrenamiento.

Finalmente, considero que sí tiene sentido que una red más profunda no aprenda mejor en este caso. Estamos utilizando varias capas con función sigmoide y MSE, y al apilar sigmoides los gradientes pueden hacerse cada vez más pequeños durante la retropropagación. Esto hace que las primeras capas aprendan muy lenta o prácticamente dejen de actualizarse. Además, Iris es un conjunto de datos relativamente pequeño y no necesariamente necesita una red tan profunda. Esto se nota bastante en las gráficas: las redes profundas reducen el error al principio, pero después sus curvas se aplanan, mientras que las redes originales consiguen seguir disminuyendo el error. En este ejercicio quedó bastante claro que agregar más capas no significa automáticamente que la red vaya a aprender mejor.

# Evidencias



```
04 Multilayer perceptron.ipynb ☆
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Comandos + Código + Texto Ejecutar todo
Mostrar paleta (Shift+P)
Red profunda

[8]
✓ 0 s
# Topology: 4 x 3 x 3 x 3 x 3

input_size_layer1 = 4
num_neurons_layer1 = 3

input_size_layer2 = num_neurons_layer1
num_neurons_layer2 = 3

input_size_layer3 = num_neurons_layer2
num_neurons_layer3 = 3

input_size_layer4 = num_neurons_layer3
num_neurons_layer4 = 3

layer1 = list()
layer2 = list()
layer3 = list()
layer4 = list()

[9]
✓ 0 s
def init_weights():
    layer1.clear()
    layer2.clear()
    layer3.clear()
    layer4.clear()

    for i in range(num_neurons_layer1):
        layer1.append(generate_weights(input_size_layer1+1))

    for i in range(num_neurons_layer2):
        layer2.append(generate_weights(input_size_layer2+1))

    for i in range(num_neurons_layer3):
        layer3.append(generate_weights(input_size_layer3+1))

    for i in range(num_neurons_layer4):
        layer4.append(generate_weights(input_size_layer4+1))
```

```
04 Multilayer perceptron.ipynb ☆
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Comandos + Código + Texto Ejecutar todo
RAM
Disco

[14]
✓ 14 s
for i in range(len(delta_layer2)):
    weighted_delta_errors = 0

    for j in range(num_neurons_layer3):
        weighted_delta_errors += layer3[j][i+1] * delta_layer3[j]

    delta_layer2[i] = output_layer2[i] * (1 - output_layer2[i]) * weighted_delta_errors

    for i in range(len(layer2)):
        layer2[i][0] += alpha * delta_layer2[i] * 1

    for j in range(len(layer2[i])-1):
        layer2[i][j+1] += alpha * delta_layer2[i] * output_layer1[j]

    delta_layer1 = np.zeros(num_neurons_layer1)

    for i in range(len(delta_layer1)):
        weighted_delta_errors = 0

        for j in range(num_neurons_layer2):
            weighted_delta_errors += layer2[j][i+1] * delta_layer2[j]

        delta_layer1[i] = output_layer1[i] * (1 - output_layer1[i]) * weighted_delta_errors

    for i in range(len(layer1)):
        layer1[i][0] += alpha * delta_layer1[i] * 1

        for j in range(len(layer1[i])-1):
            layer1[i][j+1] += alpha * delta_layer1[i] * X[d][j]

E.append( calculate_error() )

error_vec = np.asarray(E)
plt.plot(error_vec)
plt.xlabel('Epochs')
plt.ylabel('Error')
plt.show()
```

```
O4 Multilayer perceptron.ipynb
colab.research.google.com/drive/1V3CWUgD5HT_b4PMGBZygcich8Z6H#scrollTo=1pQB11m70dbb

O4 Multilayer perceptron.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Comandos + Código + Texto Ejecutar todo
RAM Desc

alpha = 0.03
epochs = 500

def init_weights():
    E = list()
    E.append( calculate_error() )
    for e in range(epochs):
        for d in range(len(X)):
            # Etapa 1 - Propagación
            output_layer1 = np.zeros(num_neurons_layer1)
            for i in range(len(output_layer1)):
                output_layer1[i] = propagate(X[d], layer1[i])

            output_layer2 = np.zeros(num_neurons_layer2)
            for i in range(len(output_layer2)):
                output_layer2[i] = propagate(output_layer1, layer2[i])

            output_layer3 = np.zeros(num_neurons_layer3)
            for i in range(len(output_layer3)):
                output_layer3[i] = propagate(output_layer2, layer3[i])

            output_layer4 = np.zeros(num_neurons_layer4)
            for i in range(len(output_layer4)):
                output_layer4[i] = propagate(output_layer3, layer4[i])

            # Etapa 2 - Retropropagación de los errores
            delta_layer4 = np.zeros(num_neurons_layer4)

            for i in range(len(delta_layer4)):
                delta_layer4[i] = output_layer4[i] * (1 - output_layer4[i]) * (Y[d][i] - output_layer4[i])

            for i in range(len(layer4)):
                layer4[i][0] += alpha * delta_layer4[i] * 1

            for j in range(len(layer4[i])-1):
```

```
O4 Multilayer perceptron.ipynb O5 Keras - multilayer perceptron - iris.ipynb
colab.research.google.com/drive/1tCW5YjCQOQVZ_Bms_XBrsDneT80PHdRw#scrollTo=W0qLqTvdK9o6

O5 Keras - multilayer perceptron - iris.ipynb
Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda
Comandos + Código + Texto Ejecutar todo
RAM Desc

idx = 0
print(X[idx])
print(target[idx])
print(Y[idx])

[5.1 3.5 1.4 0.2]
0
[1 0 0]

model = keras.Sequential(
    [
        layers.Dense(3, activation="sigmoid", name="layer1", input_shape=(4,)),
        layers.Dense(3, activation="sigmoid", name="layer2"),
    ]
)

model.summary()

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:186: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential models, prefer using an 'Input(shape)' object as the
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"

Layer (type) Output Shape Param #
layer1 (Dense) (None, 3) 15
layer2 (Dense) (None, 3) 12

Total params: 27 (108.00 B)
Trainable params: 27 (108.00 B)
Non-trainable params: 0 (0.00 B)

y = model(X)
print(y[140])

tf.Tensor([0.51487625 0.5159322 0.48963323], shape=(3,), dtype=float32)

optimizer=tf.keras.optimizers.SGD(learning_rate=0.03)
```

